



Scheduling

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Observation of Scheduling

This section explains how scheduling behavior in Linux can be observed using system tools.

1.1 Monitoring Commands

Use `top` to observe CPU usage in real time.

```
top
```

%CPU: CPU usage of the process

PR: Kernel scheduling priority

NI: Nice value (user-defined priority)

S: Process state (R = running, S = sleeping)

TIME+: Total CPU time used

COMMAND: Program name

Use `ps` to inspect scheduling-related attributes.

```
ps -eo pid,pri,ni,cls,comm
```

ps -eo: `-e` shows processes for all users, and `-o` lets you choose which columns to display.

PID: Process ID

NI: Nice value (user-defined priority), with range $[-20, 19]$, specifies how favorable a process is for CPU scheduling, where lower values (e.g., -20) indicate higher priority and more CPU time, higher values (e.g., 19) indicate lower priority and less CPU time, and the default value is 0.

PRI: Kernel scheduling priority (the actual priority used by the scheduler), with typical ranges in Linux of approximately $[100, 139]$ for time-sharing (TS) processes and $[1, 99]$ for real-time (FIFO, RR) processes (where higher values indicate higher priority), is derived from the nice value for normal processes (e.g., $PRI \approx 120 + NI$) but can also be influenced by the scheduling class and dynamic adjustments.

CLS: Scheduling class (TS for normal, FF for FIFO, RR for Round Robin, IDL for idle)

COMM: Program name

1.2 Demo

C Code Example: 01_cpu_task.c

```
1 #include <stdio.h>
2 #include <time.h>
3
4 int main() {
5     volatile unsigned long long x = 0;
6     time_t start = time(NULL);
7
8     while (time(NULL) - start < 5000) {
9         x++;
10    }
11    printf("Done: %llu\n", x);
12    return 0;
13 }
```

Terminal Commands

Execute the program, then use the following commands in another terminal to observe its scheduling behavior while it is running:

```
top
ps -eo pid,pr,ni,cls,comm | grep 01_cpu_task
```

Command Output

```
top:
PID    PR   NI   %CPU  COMMAND
16006  20   0   100.0  01_cpu_task

ps:
16006   19   0   TS  01_cpu_task
```

2 Multicore Scheduling

CPU affinity determines which core a process runs on. Processes on different cores execute in parallel without competition, while processes on the same core must share CPU time, making scheduling behavior visible. This shows how multicore scheduling and affinity affect performance and resource sharing.

Check number of logical CPUs (threads)

```
nproc
```

Run both processes on the same core and observe using **top** in a separate terminal:

```
taskset -c 0 ./01_cpu_task &
taskset -c 0 ./01_cpu_task &
```

`taskset` controls where a process runs (which CPU core), not how it is scheduled.

The CPU is shared between processes, and each process receives a portion of CPU time, typically around 50% each.

top Command Output

PID	PR	NI	%CPU	COMMAND
16308	20	0	50.1	01_cpu_task
16755	20	0	50.1	01_cpu_task

Run processes on specific cores and observe using `top`

```
taskset -c 0 ./01_cpu_task &
taskset -c 1 ./01_cpu_task &
```

Command Output

PID	PR	NI	%CPU	COMMAND	
17174	20	0	~100%	01_cpu_task	(Core 0)
17202	20	0	~100%	01_cpu_task	(Core 1)

Each process runs on its assigned CPU core, allowing them to execute independently without competing for resources. As a result, both processes can utilize their respective cores fully and reach close to 100% CPU usage at the same time.

2.1 CPU Affinity with POSIX Threads

CPU affinity can be controlled directly inside a program using POSIX functions, allowing threads to be assigned to specific CPU cores without relying on external tools such as `taskset`. The following example demonstrates how threads can be placed on different cores to enable parallel execution.

C Code Example: `02_pthread_affinity.c`

```
1 #define _GNU_SOURCE
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <sched.h>
5 #include <unistd.h>
6
7 void* work(void* arg) {
```

```
8     int core = *(int*)arg;
9     cpu_set_t cpuset;
10
11     CPU_ZERO(&cpuset);
12     CPU_SET(core, &cpuset);
13
14     pthread_setaffinity_np(pthread_self(), sizeof(cpu_set_t), &cpuset);
15
16     printf("Thread %lu bound to core %d\n", (unsigned long)pthread_self(), core);
17
18     while (1) {
19         int running_core = sched_getcpu();
20         printf("Thread ID: %lu, assigned core: %d, running on CPU: %d\n",
21             (unsigned long)pthread_self(), core, running_core);
22         sleep(1);
23     }
24
25     return NULL;
26 }
27
28 int main() {
29     pthread_t t1, t2;
30     int core0 = 0, core1 = 1;
31
32     pthread_create(&t1, NULL, work, &core0);
33     pthread_create(&t2, NULL, work, &core1);
34
35     pthread_join(t1, NULL);
36     pthread_join(t2, NULL);
37
38     return 0;
39 }
```

Run

```
gcc 02_pthread_affinity.c -o 02_pthread_affinity -lpthread
./02_pthread_affinity
```

Typical Output

```
Thread 126600563648192 bound to core 1
Thread ID: 126600563648192, assigned core: 1, running on CPU: 1
Thread 126600572040896 bound to core 0
Thread ID: 126600572040896, assigned core: 0, running on CPU: 0
Thread ID: 126600572040896, assigned core: 0, running on CPU: 0
Thread ID: 126600563648192, assigned core: 1, running on CPU: 1
Thread ID: 126600563648192, assigned core: 1, running on CPU: 1
Thread ID: 126600572040896, assigned core: 0, running on CPU: 0
Thread ID: 126600572040896, assigned core: 0, running on CPU: 0
Thread ID: 126600563648192, assigned core: 1, running on CPU: 1
```

```
Thread ID: 126600572040896, assigned core: 0, running on CPU: 0
Thread ID: 126600563648192, assigned core: 1, running on CPU: 1
```

`cpu_set_t` is a predefined structure used to represent a set of CPU cores for controlling processor affinity.

`CPU_ZERO(set)` initializes a CPU set by clearing all CPUs from it.

`CPU_SET(cpu, set)` adds the specified CPU core to the given CPU set.

`pthread_setaffinity_np(thread, cpusetsize, cpuset)` assigns a thread to a specific set of CPU cores, where `thread` is the target thread ID, `cpusetsize` is the size of the CPU set structure, and `cpuset` is a pointer to a `cpu_set_t` that defines the allowed CPU cores.

`sched_getcpu()` returns the CPU core on which the calling thread is currently executing.

Use the command below to see the threads of your program:

```
top -H -p $(pgrep -d', ' -f 02_pthread_affinity)
```

`top -H -p`: shows threads only for those PIDs, where `-H` enables thread view and `-p` restricts the output to the specified PIDs

`$(...)`: passes those PIDs to top

`-d', '` : joins them as pid1,pid2,...

`pgrep -f 02_pthread_affinity`: finds all PIDs matching your program name

Command Outputs

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26510	sareh	20	0	84612	1176	1176	S	0.0	0.0	0:00.00	02_pthread_affi
26511	sareh	20	0	84612	1176	1176	S	0.0	0.0	0:00.00	02_pthread_affi
26512	sareh	20	0	84612	1176	1176	S	0.0	0.0	0:00.00	02_pthread_affi

3 Priority Scheduling with `nice`

Linux uses priority-based scheduling for time-sharing processes, where the priority of a process is influenced by its `nice` value. A lower nice value results in a higher priority, allowing the process to receive a larger share of CPU time, while a higher nice value reduces the priority and leads to a smaller CPU share.

3.1 C Code Example: `01_cpu_task.c`

Run with Different Priorities

```
taskset -c 0 nice -n 0 ./01_cpu_task &
taskset -c 0 nice -n 10 ./01_cpu_task &
```

3.2 Observation

```
top
ps -eo pid,pri,ni,cls,comm | grep 01_cpu_task
```

Command Outputs

```
top:
PID     PR    NI   %CPU  COMMAND
17827   20     0   ~90%   01_cpu_task
17828   30    10   ~10%   01_cpu_task
```

```
ps:
PID     PRI    NI   CLS   COMMAND
17827    19     0    TS    01_cpu_task
17828     9    10    TS    01_cpu_task
```

`nice` sets the user-level priority of a process, which the kernel then translates into an internal priority value (PRI). The scheduler uses this internal priority to decide how CPU time is distributed among processes, allocating a larger share to those with higher priority.

3.3 Changing Priority of a Running Process

Step 1: Check current scheduling parameters

```
ps -eo pid,pri,ni,cls,comm | grep 01_cpu_task
```

Output:

```
24544    0   19   TS 01_cpu_task
```

Step 2: Change nice value

```
renice 10 -p 24544
```

Output:

```
24544 (process ID) old priority 0, new priority 10
```

Step 3: Verify updated values

```
ps -eo pid,pri,ni,cls,comm | grep 01_cpu_task
```

Output:

```
24544   10    9   TS 01_cpu_task
```

Explanation

The nice value (NI) changes from 0 to 10, which lowers the process priority. The kernel updates the internal priority (PRI) accordingly, while the scheduling class (TS) remains unchanged. `renice` is used to adjust the priority of a process after it has already started, which is useful when a running process is consuming too much CPU, when system responsiveness needs to be improved without stopping processes, or when priorities must be tuned dynamically based on system workload.

4 Real-Time Scheduling

Real-time scheduling has stronger priority than normal scheduling, which allows real-time tasks to preempt and interrupt normal processes. Linux provides two real-time scheduling policies: `SCHED_FIFO` and `SCHED_RR`.

4.0.1 Demo 1: FIFO Dominates Normal Scheduling

Terminal Commands

```
taskset -c 0 sudo chrt -f 50 ./01_cpu_task &
taskset -c 0 ./01_cpu_task &
top
```

`chrt` is used to assign real-time scheduling policies and priorities to processes in Linux.

`-f` selects the `SCHED_FIFO` policy, where the process runs without time slicing until it blocks or finishes.

`-r` selects the `SCHED_RR` policy, where processes of equal priority share the CPU using time slices (round-robin).

The number (e.g., 50 in `chrt -f 50`) specifies the real-time priority, with a range of [1, 99], where higher values indicate higher priority.

Command Output

PID	PR	NI	%CPU	COMMAND
27415	-51	0	~95%	01_cpu_task (FIFO)
27471	20	0	~5%	01_cpu_task (Normal)

4.0.2 Explanation

The FIFO real-time process has a higher scheduling priority than the normal process, so it is always selected by the scheduler whenever it is runnable. As a result, it receives almost all of the CPU time.

However, the normal process is not completely blocked and still executes consistently. This is primarily due to Linux's real-time throttling mechanism, which limits real-time tasks to about 95% of CPU time by default and reserves the remaining portion for non-real-time processes. Because of this enforced limit, the normal process is guaranteed a small share of CPU time even when a FIFO task is continuously runnable.

The FIFO policy is non-preemptive only with respect to other real-time processes, and while kernel activity, system interrupts, or brief transitions into the kernel may create short scheduling opportunities, these are not the main reason for the observed behavior.

Therefore, the normal process receives a small but consistent fraction of CPU time, leading to low but non-zero CPU usage. This demonstrates that real-time scheduling strongly dominates normal scheduling and can lead to near-starvation of lower-priority processes, but complete starvation is prevented in practice by the system's scheduling policy.

TODO: Repeat the experiment using `SCHED_RR` instead of `SCHED_FIFO`, what do you observe?

Answer: The behavior is similar; the real-time `SCHED_RR` process still dominates CPU usage due to its higher priority, while the normal process receives only a small share of CPU time. The main difference is that `SCHED_RR` uses time slices, but since there are no other real-time processes with the same priority, this does not significantly affect the outcome.

4.1 Demo 2: Round Robin Scheduling

Terminal Commands

```
taskset -c 0 sudo chrt -r 50 ./01_cpu_task &
taskset -c 0 sudo chrt -r 50 ./01_cpu_task &
top
```

Command Output

PID	PR	%CPU	COMMAND
28004	-51	~48%	01_cpu_task
28029	-51	~48%	01_cpu_task

4.1.1 Explanation

Both processes use the `SCHED_RR` policy with the same real-time priority, so the scheduler divides CPU time into equal time slices between them. This leads to nearly equal CPU usage for both processes, demonstrating fair sharing. Minor differences in CPU percentage are caused by measurement intervals and rapid context switching. This behavior corresponds directly to the round-robin scheduling algorithm.

4.2 Demo 3: Priority Difference in Real-Time Scheduling

Terminal Commands

```
taskset -c 0 sudo chrt -r 50 ./01_cpu_task &
taskset -c 0 sudo chrt -f 60 ./01_cpu_task &
top
```

Note 1: You can run the commands in any order; however, if FIFO is started first, preemption will not be observable. Run the RR task first and then the FIFO task to observe how the RR task gets preempted.

Note 2: If you run RR first, when running `top`, both processes may be visible for a short time after they are started (as shown below). This is because the lower-priority RR process can still execute briefly due to system activity such as interrupts or kernel operations, which creates small gaps in CPU usage.

Command Output

PID	PR	%CPU	COMMAND
18688	-51	~15%	01_cpu_task (RR, priority 50)
18743	-61	~80%	01_cpu_task (FIFO, priority 60)

After a short time, the higher-priority FIFO process continuously occupies the CPU, reaching up to about **95%** due to the **real-time scheduler throttling limit**. As a result, the RR process receives almost no CPU time and may disappear from the display, since `top` typically shows only the most active processes.

To ensure that both processes remain visible regardless of their CPU usage, you can restrict `top` to them explicitly:

```
top -p $(pgrep -d', ' -f 01_cpu_task)
```

Command Output

PID	PR	%CPU	COMMAND
19541	-61	95%	01_cpu_task (FIFO, priority 60)
19520	-51	0%	01_cpu_task (RR, priority 50)

4.2.1 Explanation

The FIFO process has a higher real-time priority than the Round Robin process, so it is always selected by the scheduler whenever it is runnable. In real-time scheduling, priority is the primary decision factor: a higher-priority task will preempt any lower-priority task, regardless of whether the lower-priority task uses FIFO or Round Robin.

FIFO being non-preemptive means that it will not be preempted by *other tasks of equal priority*. However, it can still preempt lower-priority tasks, and it may also temporarily stop running due to system events such as interrupts or kernel activity. During these short intervals, the CPU becomes available, and the lower-priority RR task may run briefly. In addition, Linux enforces real-time scheduler throttling, which limits the total CPU time available to all real-time

tasks to about 95% over a scheduling period, further restricting the execution opportunities of lower-priority real-time tasks.

As a result, the FIFO process receives most of the CPU time, while the RR process runs only occasionally and therefore shows much lower CPU usage. The %CPU values shown by `top` are approximate and based on sampling over short time intervals, so they may not sum exactly to 100%. For example, the RR process may still receive brief execution windows, leading to values such as ~15% and ~80%, which do not add up precisely due to rounding and measurement effects. In some cases, the FIFO process may appear to use 100% CPU while the RR process shows 0%, because the sampling interval of `top` does not capture the small portion of time reserved for non-real-time tasks. Over a longer period, however, the FIFO process remains limited to about 95% CPU due to real-time throttling. These values can also vary between runs, since scheduling behavior depends on timing, system activity, and measurement intervals, as well as the effects of real-time bandwidth limits.

4.3 Real-Time Scheduling with POSIX Threads

POSIX provides functions that allow programs to directly control and inspect the scheduling policy and priority of threads at runtime. These functions offer a programmatic interface to scheduling, serving as the equivalent of using tools such as `chrt` from the terminal, but integrated within the application itself. The following example demonstrates how to create a thread with a specific real-time scheduling policy and priority using the Pthreads API. By selecting different policies, such as `SCHED_RR` or `SCHED_FIFO`, it is possible to observe and compare real-time scheduling behaviors directly in code, corresponding to the same concepts explored through command-line tools.

C Code Example: `03_pthread_sched.c`

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <sched.h>
4 #include <string.h>
5
6 void* work(void* arg) {
7     int policy;
8     struct sched_param param;
9
10    if (pthread_getschedparam(pthread_self(), &policy, &param) == 0) {
11        if (policy == SCHED_FIFO)
12            printf("Thread policy: SCHED_FIFO, priority: %d\n", param.↵
                sched_priority);
13        else if (policy == SCHED_RR)
14            printf("Thread policy: SCHED_RR, priority: %d\n", param.↵
                sched_priority);
15        else
16            printf("Thread policy: OTHER, priority: %d\n", param.↵
                sched_priority);
17    }
```

```
18
19     volatile unsigned long long x = 0;
20     while (1) {
21         x++;
22     }
23     return NULL;
24 }
25
26 int main() {
27     pthread_t t;
28     pthread_attr_t attr;
29     struct sched_param param;
30
31     pthread_attr_init(&attr);
32
33     pthread_attr_setschedpolicy(&attr, SCHED_RR);
34     param.sched_priority = 50;
35     pthread_attr_setschedparam(&attr, &param);
36     pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
37
38     int ret = pthread_create(&t, &attr, work, NULL);
39     if (ret != 0) {
40         printf("pthread_create failed: %s\n", strerror(ret));
41         return 1;
42     }
43
44     pthread_join(t, NULL);
45     return 0;
46 }
```

Run

```
gcc 03_pthread_sched.c -o 03_pthread_sched -lpthread
sudo ./03_pthread_sched
```

Typical Output

```
Thread policy: SCHED_RR, priority: 50
```

`sched_param` is a POSIX-defined structure that contains scheduling parameters, and in Linux it primarily includes the field `sched_priority`, which specifies the priority level for real-time scheduling policies.

`pthread_getschedparam(thread, policy, param)` retrieves the scheduling information of a thread, where `thread` specifies the target thread ID, `policy` is a pointer that receives the scheduling policy (such as `SCHED_FIFO` or `SCHED_RR`), and `param` is a pointer to a `sched_param` structure that receives the thread's priority.

`pthread_attr_setschedpolicy(attr, policy)` sets the scheduling policy of a thread attribute object, where `attr` is a pointer to the thread attributes and `policy` specifies the scheduling policy.

`pthread_attr_setschedparam(attr, param)` sets the scheduling parameters of a thread attribute object.

`pthread_attr_setinheritsched(attr, inherit)` controls whether a thread inherits its scheduling attributes from the parent or uses explicitly defined attributes, where `inherit` is either `PTHREAD_INHERIT_SCHED` or `PTHREAD_EXPLICIT_SCHED`.

5 Thread Scheduling (Optional)

In Linux, scheduling is performed on kernel-level threads rather than processes.

Scheduling can occur in process-contention scope (PCS), where threads compete within a process, or system-contention scope (SCS), where all threads compete globally. Linux uses only SCS.

Single-Threaded Program: [04_single_thread.c](#)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 int main() {
4     volatile unsigned long long x = 0;
5     printf("Thread ID: %lu\n", (unsigned long)pthread_self());
6     while (1) {
7         x++;
8     }
9     return 0;
10 }
```

Run and observe:

```
./04_single_thread
ps -elf | grep 04_single_thread
```

LWP (Lightweight Process) represents a kernel-level thread, which is the actual schedulable unit managed independently by the Linux scheduler.

The program has only one thread, so it appears as a single schedulable entity. The number of threads (**NLWP**) is 1.

Multi-Threaded Program: [05_multi_thread.c](#)

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void* work(void* arg) {
5     volatile unsigned long long x = 0;
6     printf("Thread ID: %lu\n", (unsigned long)pthread_self());
7     while (1) {
8         x++;
9     }
10    return NULL;
11 }
12 int main() {
13     pthread_t t1, t2;
14     pthread_create(&t1, NULL, work, NULL);
15     pthread_create(&t2, NULL, work, NULL);
16     pthread_join(t1, NULL);
```

```
17     pthread_join(t2, NULL);
18     return 0;
19 }
```

Run and observe:

```
./05_multi_thread
ps -eLf | grep 05_multi_thread
```

Multiple threads appear as separate entries with the same process ID but different thread IDs (**LWP**). Each thread is scheduled independently by the kernel.

Competition on a Single Core

```
taskset -c 0 ./05_multi_thread
top -H
```

-H in **top** enables thread view, displaying each kernel thread as a separate entry instead of aggregating them under a single process.

When all threads are restricted to the same CPU core, they cannot run in parallel and must share CPU time. The scheduler alternates execution between them, resulting in each thread receiving approximately half of the CPU. This demonstrates competition among threads on a single core.

Parallel Execution on Multiple Cores

```
taskset -c 0,1 ./05_multi_thread
top -H
```

When threads are allowed to run on multiple CPU cores, they execute in parallel. Each thread runs on a separate core and can reach close to 100% CPU usage independently.

Can We Change Thread Scheduling Scope?

What will happen if we set `PTHREAD_SCOPE_PROCESS`? Will threads behave differently?

C code example: 06_scope.c

```
1 #include <pthread.h>
2 #include <stdio.h>
3 int main() {
4     pthread_attr_t attr;
5     int scope;
6     pthread_attr_init(&attr);
```

```
7   pthread_attr_getscope(&attr, &scope);
8   printf("Initial scope: %s\n",
9         (scope == PTHREAD_SCOPE_SYSTEM) ? "SYSTEM" : "PROCESS");
10
11   int ret = pthread_attr_setscope(&attr, PTHREAD_SCOPE_PROCESS);
12   if (ret != 0)
13       printf("Setting PROCESS scope failed\n");
14   pthread_attr_getscope(&attr, &scope);
15   printf("Final scope: %s\n",
16         (scope == PTHREAD_SCOPE_SYSTEM) ? "SYSTEM" : "PROCESS");
17   return 0;
18 }
```

`pthread_attr_init` initializes a thread attribute object with default settings so it can be used safely.

`pthread_attr_getscope` retrieves the current contention scope (PROCESS or SYSTEM) from a thread attribute object.

`pthread_attr_setscope` attempts to set the contention scope of a thread attribute object to either PROCESS or SYSTEM.

Typical Output

```
Initial scope: SYSTEM
Setting PROCESS scope failed
Final scope: SYSTEM
```

What Happened?

Even though the API allows selecting between PROCESS and SYSTEM scope, Linux supports only system-contention scope (SCS), meaning all threads compete globally for CPU time. Attempts to switch to PROCESS scope fail, illustrating that not all theoretical or API-defined features are implemented in practice, and thread scheduling in Linux is always system-wide.